*Back to index*

# Query Evaluation Techniques for Large Databases (5,8)

**Goetz Graefe**

*Summary by: Steve Gribble and Armando Fox*

**One-line summary:** Discussion of the implementation, performance and optimizations for various kinds of join algorithms, and a few words on query scheduling to minimize I/O cost.

## Overview/Main Points

### Join algorithms

- Most systems today use nested-loop join or merge join, since System R found that these always gave the best or close to the best performance. *Hash joins* have recently been studied with some interest, though.
- **Nested-loop join:** scan "inner" input once for each value of "outer" input, looking for a match. Some optimizations to alleviate the horrible performance:
  - For one-to-one match, can terminate inner scan as soon as match found.
  - Scan inner input once for each *block* (page) of outer input (block nested-loop join).
  - Try to save some pages of inner input in memory.
  - Scan inner input alternately backwards and forwards, to reuse the last page or two of inner input in memory.
- **Indexed nested loop join** exploits a temporary or permanent index on the inner input's join attribute.
  - Can be way fast under certain conditions (index depth * size of smaller input < size of larger input).
  - "Zig-zag join" requires an index on both inputs; alternates scanning for a join value in one and then looking it up in the other.
- **Merge join**: both inputs sorted on join attribute, then scanned by stepping a pair of pointers. Efficient if the inputs are produced by a previous merge-join step, since that means they're already sorted.
  - Heap filter merge join: use temporary memory to create sorted runs from outer input using replacement selection; join immediately with sorted inner input; repeat.
- **Hash join**: Build in-memory hash table on one input, probe it with items from the other.
  - If inputs too large, they're recursively partitioned. Outputs from each partitioning are concatenated.
  - Cost is roughly logarithmic in input size; main difference compared to merge join is that the recursion depth depends only on the build input, not both inputs.
- **Pointer-based join:** Used to gather a set of tuples S pointed to by pointers from tuples in R.
  - Nested-loops variant: scans through R and retrieves the appropriate S tuple for each R tuple. All the same performance problems as regular nested-loops join.
  - Merge-join variant: sort R on pointer values, make a single pass over the disk to read all tuples pointed to.
  - Hybrid hash join variant: collects together R tuples containing pointers to same S page, then scans.
  - Pointer based joins typically outperform value joins if only a small fraction of the outer input actually participates in the join.
- High order bit: good cost functions are needed so that the query optimizer can decide what flavor of join to use.

### Optimal query scheduling

- Used to be unimportant, since all execution plans were left-deep trees and sorting to disk was done at each intermediate step.
- Today's plans include bushy (ie parallelizable) execution trees and extensive pipelining of successive steps, so scheduling matters.
- **Optimal buffer management**: map disk pages to buffers so as to avoid the largest number of I/Os in the future.

    - Cost functions exhibit "steps" when plotted over available buffer space; should allocated buffer at low end of a step.
    - Separate page replacement algorithm for each scan allow finer control of buffer allocation.
- **Natural stopping points:** points at which all data are in temporary/intermediate disk files. A good time to switch efficiently between independent subplans.
- Hybrid-hash and similar binary match operations should allow overflow avoidance as a runtime option, to allow the query optimizer to insert a stop point.
- Binary-operator implementations should have a switch controlling which subplan (left or right) is initiated first.
- Memory should be proportionally divided among multiple concurrently-active operators.
- For recursive hybrid hash joins, finish level N-1 before starting level N. Reason: if level N consumes results immediately as they are generated by N-1, you do get some pipelining but you also get the two levels competing for memory.
- Allocation of resources other than memory "is an open issue that should be addressed soon".
- Scheduling bushy trees for concurrent execution is not well understood. Something like Ingres decomposition is likely to yield better results in practice than trying to produce optimal scheduling. (Aren't most flavors of optimal scheduling NP-complete anyway?)

# Relevance

I/O reduction is the name of the game; nested loop, merge, and hash join algorithms are I/O intensive, so optimizations for them (as well as optimal query scheduling, which matters now a lot more than it used to) are a bunch of hacks to reduce I/O costs.

# Flaws

- The author should summarize conclusions somewhere (at the beginning or the end). The sentence above sums up the whole point of these sections.
- So boring to read that my teeth were falling asleep, but I guess that's not too surprising for a survey paper.

---

*Back to index*